

Data pre-processing

We performed a retrospective study on incident attendance times of the London Fire Brigade between the years 2009 and 2024.

Data pre-processing was performed by (1) researching background information on the variables within each table and by (2) running descriptive statistics on the dataset to obtain an overview of the data structure of each variable.

At first, the tables were uploaded as dataframes. Next, we merged these into a single dataframe based on incident numbers and call times. The subsequent data pre-processing steps involved:

- A conditional imputation of delay code descriptions.
- A calculation of the distance from the fire station to the incident location based on the coordinate differences between incident location and fire station.
- A replacement of all turnout times smaller than 10 s with the mean turnout time.
- A replacement of all travel times smaller than 15 s with the mean travel time.
- A replacement of all attendance times smaller than 25 s (10+15) with the mean attendance time.
- A replacement of all NaNs in second pump arriving time with -1 and
- A replacement of all NaN in second pump deployed from station with -1.

We defined the attendance time (in seconds) as our **target** and dropped all variables that are non-predictive (Incident ID), are part of attendance time or show little correlation with the target variable.

Prior to modelling, all categorical data were encoded via frequency encoding or OneHot encoding, depending on the cardinality of categorical variables.

After encoding, the data was split into a train and a test sample and different modeling approaches were tested to predict attendance times as reliably as possible. The test sample included 30% of the data.

1. MLPRegressor

Initially, we used a relatively small model in order to limit execution time (hidden_layer_sizes=(10, 5), max_iter=10), then we optimized (other) hyperparameters using GridSearchCV.

Selected hyperparameter-combinations were:

solver	lbfgs, adam, sgd
activation	relu, tanh, logistic
alpha	0.0001, 0.001

The best hyperparameters are printed in **bold**. They were used for all future MLP-models.

For all MLP-models, we calculated the mean absolute error (MAE), root mean squared error (RMSE) and the model score (R^2). To test for overfitting, we calculated these 3 metrics for

both the train set and the test set. We found that the differences between the train and test sets are minimal across all 3 metrics (MAE, RMSE, R^2). This suggests that the models generalize well to unseen data, showing no signs of overfitting.

We started with

hidden_layer_sizes=(10,10), max_iter=30,

then increased the number of neurons (hidden_layer_sizes) and (maximum) iterations (max_iter) to see if it improves results.

The following table shows the results. Values are rounded to 4 digits.

set (train/test)	hidden_layer_sizes	max_iter	MAE	RMSE	R^2
test	(10,10)	30	73.6582	107.3482	0.4982
test	(20,20)	100	72.7414	105.9014	0.5118
train	(10,10)	30	73.6073	107.135	0.4991
train	(20,20)	100	72.6529	105.6844	0.5126

The improvements that come with increasing the number of neurons and iterations are marginal. Therefore, we decided against a further increase in the number of neurons and (max) iterations. However, we adjusted the learning rate, leaving hidden_layer_sizes and max_iter constant ((20,20) and 100, respectively).

The default learning rate is 0.001.

First, we tried increasing the learning rate, while leaving the number of neurons and (max) iterations constant. This led to worse results, as the following table demonstrates.

set (train/test)	learning_rate_init	MAE	RMSE	R^2
test	0.005	112.3708	151.5866	-0.0004
test	0.002	73.8362	107.3254	0.4985
train	0.005	112.285	151.3971	-0.0003
train	0.002	73.7273	107.039	0.5

Due to the fact that increasing the learning rate resulted in worse results, we next decreased the learning rate, using GridSearchCV to find the best learning rate:

Within GridSearchCV, we tested the following learning rates

- (default),
- 0.00033,
- 0.0001,
- 0.000033,
- 0.00001,
- 0.0000033 and
- 0.000001.

We also gave the options of constant or adaptive learning rate.

A constant learning rate of 0.00001 (1/100 of the default) gives the best R^2 (as determined by GridSearchCV, applied to training data).

The following table shows the metrics of the corresponding MLP-model:

set (train/test)	MAE	RMSE	R^2
test	68.626	101.091	0.5551
train	68.4785	100.6834	0.5576

Notably, this table gives the best metrics across all MLP Regressors we tried.

2. XGBoost

Next, we experimented with XGBoost, which we tested in 2 stages:

1. an initial default configuration and
2. an optimized version using hyperparameter tuning.

Initially, we trained an XGBoost regressor using its default hyperparameters to establish a baseline performance. The results for the default model were:

MAE	69.54
MSE	10431.16
R^2	0.55

While the default model provided reasonable results, we sought to improve its performance using hyperparameter tuning. To enhance the model's predictive ability, we performed a hyperparameter search using RandomizedSearchCV with 5-fold cross-validation.

After running the search, the best-performing hyperparameters were identified as:

- subsample: 1.0,
- reg_lambda: 0,
- reg_alpha: 0.1,
- n_estimators: 300,
- max_depth: 8.
- learning_rate: 0.2,
- colsample_bytree: 0.6

The tuned XGBoost model demonstrated improved performance with the following evaluation metrics:

MAE	67.25
MSE	9939.09
R^2	0.57

These results indicate that hyperparameter tuning significantly enhanced the model's predictive accuracy compared to the default model.

3. Elastic Net Regularization

Next, attendance times were predicted based on Elastic Net Regularization. Depending on the structure of the data, the train set was scaled by either Robust Scaler, Min Max Scaler or Standard Scaler.

This model was chosen due to its capacity to reduce overfitting and to handle multicollinearity. We initially trained an elastic net model with an L1 ratio of 0.5 and no alphas.

This model obtained the following results:

RMSE Train:	144.8
RMSE Test:	145.1
Train score:	0.084
Test score:	0.084

We then applied an iterative fitting along the regularization path to find the best parameters and fine-tuned the model according to the best alpha score and L1 ratio.

The optimized model produced the following results:

RMSE Train:	105.3
RMSE Test:	105.6
Train score:	0.52
Test score:	0.52

These results demonstrate that hyperparameter tuning significantly improved model predictions. However, model performance after hyperparameter tuning remains rather modest.

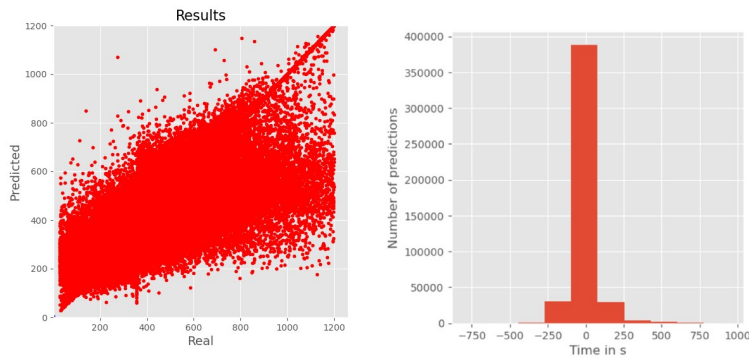
4. Random Forest Regressor

Next, attendance times were predicted using the Random Forest Regressor. Data scaling was performed as previously described, because Random Forests –despite being based on decision trees – are susceptible to outliers and imbalanced data, both of which characterize our dataset.

The initial model tested showed the following results:

MSE:	6182.623
RMSE:	78.630
R ² score:	0.731

The figure below (left panel) plots the predicted attendance times versus the real attendance times. The figure on the right depicts the error of predicted attendance times.



Next, we tested a hyperparameter tuning by applying a grid search. Grid search parameters were:

n_estimators	100, 300
max_depth	10, 20
min_samples_split	5 , 20
min_samples_leaf	2, 6

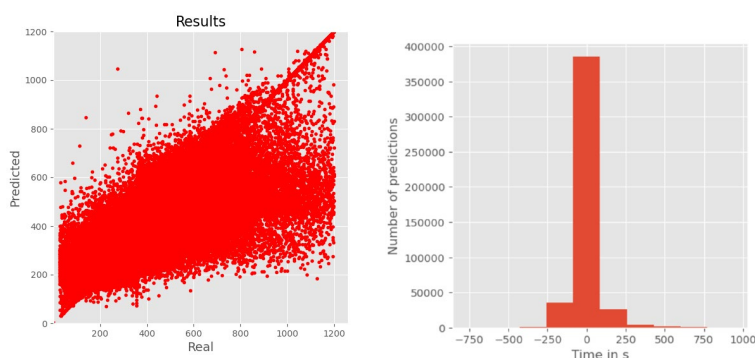
The best parameters detected by the grid search are printed in **bold**.

The corresponding Random Forest Regressor with optimized hyperparameters showed slight improvements, with an increased the R^2 score and a reduced error score.

Specifically, the following results were obtained after hyperparameter tuning:

MSE:	6189.393
RMSE:	78.673
R^2 score:	0.730

The corresponding plots are depicted below:



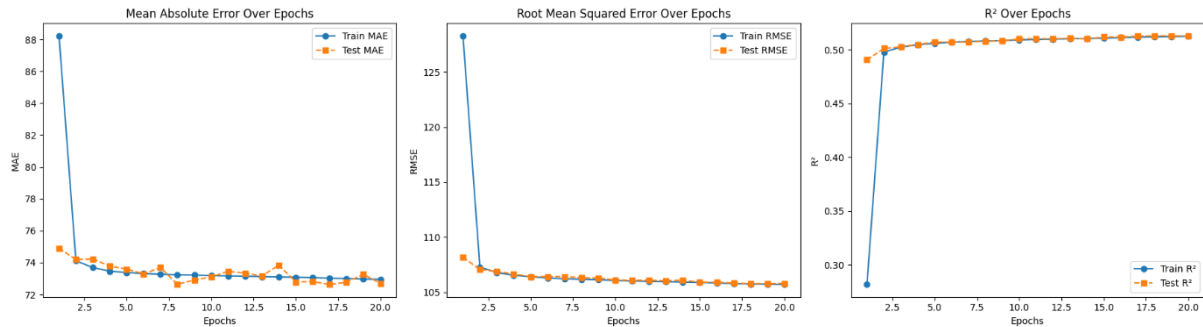
Given that the hyperparameter search took nearly 12 hours, there was little room for further improvements. Indeed, individual modifications of Random Forest Parameters (e.g. increasing the number of trees from 300 to 500) showed no further improvements.

In summary, these results demonstrate that Random Forest Regressor represents a good method to predict arrival times based on the data used in this project.

5. Neural Networks

Finally, we tested a prediction of attendance times with Neural Networks. At first, we created a simple model with just one hidden layer containing 5 neurons. Thus, we could train with a relatively large number of epochs (20) while keeping the computation time reasonable. We observed diminishing improvements after about 3 epochs, indicating that the model was already learning most of the relevant patterns early.

The following diagrams show the results over the Epochs:

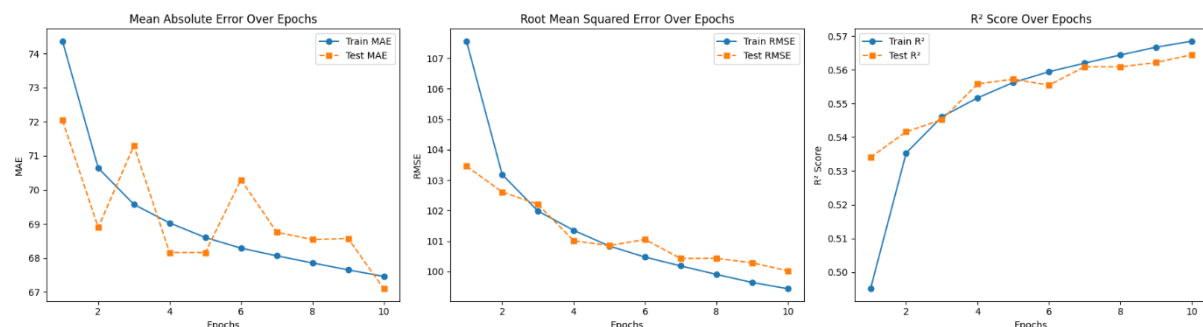


The following table gives our metrics (MAE, RMSE, R^2 [score]) after 20 epochs:

	MAE	RMSE	R^2
test set	72.703	105.7998	0.5127
train set	72.9484	105.6971	0.5124

Next, we tested whether a more complex neural network is able to improve performance. Therefore, we created a model featuring two hidden layers, with 200 neurons each. Given our earlier findings—diminishing improvements with increasing epochs—we reduced the number of epochs to 10. This approach ensures efficiency while preserving a “reserve” of epochs should the model’s increased complexity necessitate further training.

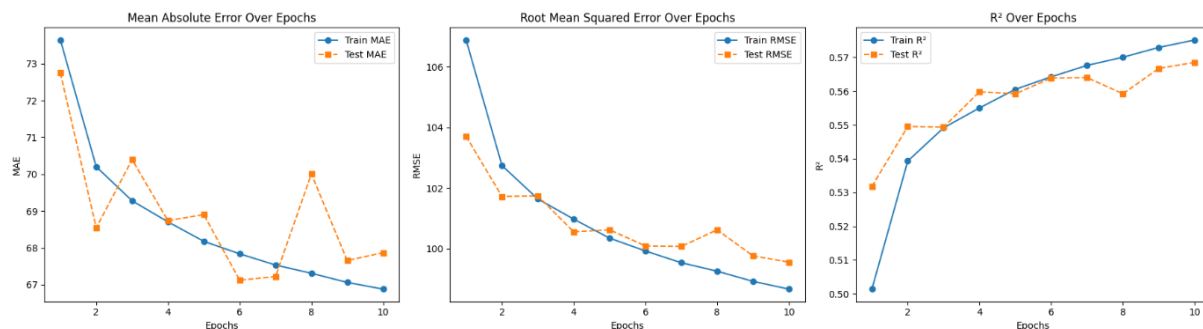
Again, we show the diagrams of how the metrics evolve over the epochs and the final metrics after 10 epochs:



	MAE	RMSE	R^2
test set	67.1001	100.0212	0.5645
train set	67.4609	99.4354	0.5685

The performance of that complex network (2 hidden layers, 200 neurons each) is slightly superior to the simpler network (1 hidden layer, 5 neurons each).

Next, we created a model with 3 hidden layers and 500 neurons each (and again, 10 epochs) to see if that improves the performance further, leading to the following results:



	MAE	RMSE	R ²
test set	67.8688	99.5571	0.5685
train set	66.8813	98.668	0.5751

Adding even more neurons (200 -> 500) and a third hidden layer did not increase the performance. Thus, we stopped here and did not create even more complex neural networks.

Our last model already took approx. 1:30 hours to execute.

Summary and Conclusion

We tested several models to predict attendance times of the London Fire Brigade (LFB). Overall, all models tested show limited predictive power. The best root mean squared error (RMSE) achieved is 89 [seconds]. Given that the mean attendance time is 355 seconds (approx. 6 minutes) with a standard deviation of 15 seconds, this error margin is rather large.

Nevertheless, the model does outperform the naive baseline of simply predicting the mean, indicating that the selected variables do indeed influence attendance time. However, their impact is overshadowed by randomness. In other words, while the model captures some meaningful patterns, chance remains a dominant factor in determining outcomes.

In addition, we demonstrate that hyperparameter tuning is critically important, as it resulted in an improvement of all but one models tested.

Reasons for the rather modest predictive power of all models are complex. In the following, we list 3 reasons that may complicate attendance time predictions: (1) A major reason that might have impacted model performance is raw data input. The raw datasets contain a lot of missing values and/or incorrect data inputs. Therefore, (i) a detailed research on variables within the datasets and (ii) extensive data pre-processing steps were necessary. To this end, we made an inquiry about specific variables to the LFB 5 weeks ago, however, we only received a reply this week (i.e. after data pre-processing). The data pre-processing steps involved a conditional imputation of delay code IDs and a data cleaning of attendance times, as this variable contained a lot values that we considered impossibly short. This data cleaning step of attendance times might have had an effect on model performance. (2) Another reason

for poor predictive performance is the fact that many more variables that have not been recorded influence attendance times. Among these are: 1) bank holidays, 2) LFB staff (sickness leaves, LFB strikes), 3) renovations of LFB fire stations, 4) streets travelled (i.e. small road vs. ring road). **(3)** Lastly, the observation period is rather long (between 2009 and 2024) and the LFB but also London traffic has undergone considerable changes within this period. In addition, some major events (e.g. Summer Olympics in 2012 or COVID pandemic) took place within this period, leading large variations within the dataset.

We therefore conclude that modeling of LFB attendance times might perform better with fewer but very robust observations.